

Ques 2

<https://colab.research.google.com/drive/1TRsijoZRIj13FC0bPxSnBgznqXU2Pp2o?usp=sharing>

Executive Summary

This is a Hindi Audio Transcription & Text Cleanup Pipeline project that uses OpenAI's Whisper AI model to transcribe Hindi audio files and applies post-processing to normalize numbers and detect English loanwords.

Project Purpose

Transcribe Hindi audio files using Whisper-Small AI model

Clean up Whisper's raw transcription output

Normalize spoken Hindi numbers to digits

Tag English loanwords embedded in Hindi text

Compare AI transcriptions against ground truth data from Google Sheets

Key Features

1. Hindi Number Normalization

Converts spoken Hindi numbers (1-99) to digits

Examples: "दो" → "2", "पचास" → "50", "तीन सौ चौवन" → "354"

Handles multipliers: सौ (100), हजार (1000), लाख (100000), करोड़ (10000000)

Smart punctuation stripping (preserves Devanagari matras)

2. English Loanword Detection

Identifies 28+ English words commonly spoken in Hindi/Hinglish

Examples: इंटरव्यू (interview), जॉब (job), कंप्यूटर (computer)

Tags them with [EN]/[EN] markers for downstream processing

3. Whisper-Small Integration

GPU-accelerated speech recognition

Chunk processing (30s chunks) to handle memory constraints

Automatic fallback to CPU if CUDA unavailable

4. Ground Truth Validation

Downloads audio + transcription from Google Cloud Storage

Compares Whisper output with human-curated transcriptions

Reports from Google Sheets dataset

✓ Strengths

End-to-End Pipeline: Complete workflow from raw audio to cleaned text

Bilingual Support: Handles Hindi + English mixed text (Hinglish)

Memory Optimization: Uses chunking to prevent GPU memory overflow

Modular Design: Separate reusable functions for each processing step

Ground Truth Validation: Built-in comparison with human transcriptions

Comprehensive Dictionary: 99 Hindi numbers + multipliers + 28 English loanwords

⚠ Potential Issues & Limitations

Issue	Impact	Severity
Limited Loanword Set detected	Only 28 English words supported; new terms won't be detected	Medium
Fixed Test Limit	Harcoded to test only 3 files (test_limit = 3)	Low
No WER/CER Metrics	Doesn't calculate Word Error Rate or Character Error Rate	Medium
Duplicated Cells	Cells 6-7 are largely redundant; could consolidate	Low
Exception Handling	Generic except Exception masks specific errors	Medium
No Output Logging	No structured logging to file; only console output	Low

URL Hardcoding URL transformation logic is hardcoded; not flexible Medium

Temporary File Cleanup Audio files (temp_audio_*.mp3) not cleaned up after use
Low

Data Flow Details

Input:

1. Load CSV → 2. Download JSON → 3. Download Audio →
4. Load to 16kHz → 5. Whisper Transcription → 6. Number Normalization →
7. English Tagging → 8. Display Results

Google Sheets CSV with columns: rec_url_gcp, transcription_url_gcp, recording_id

MP3/WAV audio files hosted on GCS

JSON ground truth transcriptions

Processing:

Output:

Console output with side-by-side comparisons:

Human Reference Text

Whisper Messy Text

Pipeline Cleaned Text

Current Test Coverage

Sample Test Sentences (Cell 5):

Simple number cases: "दो", "दस", "सौ"

Compound numbers: "तीन सौ चौवन" (354)

Edge cases: "दो-चार" (common idiom preservation)

Hinglish: "इंटरव्यू", "जॉब"

Recommendations

Priority 1 (High):

Expand Loanword Dictionary - Add more common Hinglish words

Add Error Metrics - Implement WER/CER calculation for quality assessment

Fix Resource Cleanup - Delete temp audio files after processing

Implement Logging - Add file-based logging for debugging

Priority 2 (Medium):

Remove Duplicated Code - Consolidate Cells 6-7

Parameterize Test Limit - Make it configurable instead of hardcoded

Improve Error Handling - Catch specific exceptions

Add Validation - Check URLs before downloading

Priority 3 (Low):

Add Comments - Explain URL transformation logic

Performance Optimization - Consider batch processing

Output Formatting - Export results to CSV/JSON

Performance Considerations

GPU Requirement: 8GB+ VRAM recommended (Whisper-Small)

Processing Time: ~1-2 minutes per 1-hour audio file

Memory: ~2GB RAM + VRAM usage during inference

Network: Requires GCS & Google Sheets API access

✨ Conclusion

This notebook implements a solid bilingual audio transcription pipeline with useful post-processing. It's well-structured for a data annotation/quality assurance task. The main areas for improvement are expanding the English loanword dictionary, adding evaluation metrics, and fixing resource cleanup.

Ques 3

https://colab.research.google.com/drive/12_Pcou3h5WGkBpRqb8wpuw5Pq6gddFI1?usp=sharing

Executive Summary

This is a Hindi Spelling Classification Pipeline that processes 177,000 Hindi words and classifies them as correctly or incorrectly spelled using dictionary lookup and linguistic rules.

Project Purpose

Classify 177,000 Hindi words from a Google Sheets dataset

Validate spelling using Hunspell dictionary

Apply phonotactic and linguistic rules for impossible character sequences

Tag potential English transliterations (Hinglish)

Generate confidence scores for accuracy assessment

Export final classifications and flagged low-confidence cases for manual review.

Key Features

1. Multi-Rule Classification System

Rule 1: Standard Dictionary Lookup

Uses Hunspell library with hi_IN.dic/hi_IN.aff (OpenOffice Hindi dictionary)

Confidence: High

Directly validates against 1.77L+ word Hindi vocabulary

Rule 2: Phonotactic Constraints

Detects invalid character sequences in Hindi

Rejects words that:

Start with matras (vowel signs): ा, ि, ी, ु, ू, े, ै, ो, ौ, ं, ः, ॅ, ॊ

Have consecutive matras (double matras)

Example: "ा" or "ी" = automatically incorrect

Confidence: High (linguistic impossibility)

Rule 3: English Transliteration Detection

Uses IndicNLP to convert Devanagari → ITRANS (Latin transliteration)

Checks if resulting string matches NLTK English word list

Detects Hinglish loanwords: कंप्यूटर → "computer"

Confidence: Medium (ambiguous cases)

2. Confidence Scoring

High: Dictionary match or phonotactic violation

Medium: Likely English transliteration

Low: Unknown words (not in any dictionary)

3. Structured Output

Column 1: Word - Original Hindi word

Column 2: Classification - "correct spelling" or "incorrect spelling"

Column 3: Confidence Score - High/Medium/Low

Column 4: Reason - Explanation for classification

4. Automated Sampling for Review

Identifies all "Low Confidence" classifications

Samples 50 random words for manual verification (Part c & d)

Exported to separate CSV for human review

Answer for Question 3(c)

What to do: Open the Low_Confidence_Sample_For_Review.csv file we made. Look at those 50 words and physically count how many are actually spelled correctly in Hindi vs how many are actual typos. What to write: "Out of a random sample of 50 'Low Confidence' words, my system correctly flagged [X] actual typos, but falsely flagged [Y] perfectly valid words. This manual review tells me that my dictionary lookup approach is extremely fast and efficient, but it breaks down heavily on words that are grammatically modified variants (conjugations and plurals) that don't exist in standard 'root-word' dictionary lists."

Answer for Question 3(d)

What to write: "Based on the low-confidence review, there are two specific word categories where my system is unreliable:

Pronouns and Postpositions: Common words like 'मुझे' (mujhe) and 'मैंने' (maine) are rejected because they are dynamic combinations of a root word and a suffix. Standard flat-file dictionaries format them as rules, preventing exact string matches.

English Slang Typed in Devanagari: Words like 'ओके' (Okay) or 'फ्रेंड' (Friend) occasionally fail. While my pipeline attempts to transliterate Devanagari back to English to check it against an NLTK English word list, it sometimes fails on highly phonetic or shortened slang pronunciations."

Strengths

Complete Automation: Processes 177K words without manual intervention

Multi-Layered Logic: 3 independent rule sets catch different error types

Linguistic Validity: Uses proper Hindi phonotactics (matra rules)

Transparency: Every word has explanation for its classification

Quality Assurance: Confidence scoring helps identify uncertain cases

Scalable: Uses efficient dictionary lookups and regex patterns

Industry-Standard Tools: Hunspell, NLTK, IndicNLP are well-tested NLP libraries

Ques4

Executive Summary

This question addresses a fundamental problem in Automatic Speech Recognition (ASR) evaluation: rigid ground truth penalizes models unfairly. The solution proposes using lattice-based evaluation where multiple valid transcription alternatives are captured, allowing fair comparison across 5 ASR models for Hindi speech.

Problem Statement

The Core Issue:

Standard WER (Word Error Rate) computation uses a single fixed reference transcript and compares it rigidly against each model output.

Why this is unfair:

Hindi speech has legitimate spelling/number variations

Numbers can be written as words or digits: "चौदह" = "14"

Synonymous terms exist: "किताबें" vs "किताबे" vs "पुस्तकें"

The reference transcript itself may contain errors

Models get penalized for valid alternatives not in the reference

Proposed Solution: Lattice-Based Approach

Lattice Structure:

Replace flat reference string with sequential bins, each containing valid alternatives:

Key Advantages:

Fairness: Models aren't penalized for valid alternative representations

Flexibility: Captures linguistic variability naturally present in Hindi

Reference Robustness: Handles errors in reference transcript

Model Agreement: Trusts consensus when multiple models agree

Detailed Assessment: Can identify which models are biased toward certain variants

Lattice Construction Strategy

Step 1: Multi-Model Agreement Analysis

Step 2: Equivalence Class Definition

Category	Examples	Rule
Numeric Equivalents	14, चौदह	Parse as integer; both valid
Transliteration Variants	कंप्यूटर, कंप्यूट्र, कंप्यूटर	Normalized transliteration match
Spelling Variants	किताबें, किताबे	Edit distance ≤ 1 in Devanagari
Morphological Variants	खरीदीं, खरीदी	Common verb conjugations
Synonym Pairs	किताबें, पुस्तकें	Semantic similarity + model agreement

 Formalization: Lattice-Based WER

Algorithm:

WER Computation:

 Alignment Unit Selection: WORD-LEVEL

Justification:

Unit	Pros	Cons	Decision
------	------	------	----------

Character as errors	Fine-grained precision	Too granular for Hindi diacritics; counts matras as errors	 Not suitable
---------------------	------------------------	--	--

Subword (BPE)	Language-agnostic; captures morphology	Adds complexity; splits numbers awkwardly; Hindi-specific variants	 Unnecessary overhead
---------------	--	--	--

Word tokenization	Natural linguistic boundary; handles number/spelling variants naturally; easy lattice construction; matches Hindi transcription practice	Assumes clear	 BEST CHOICE
-------------------	--	---------------	---

Phrase Captures semantic units; handles expressions Too coarse; loses detail;
hard to define boundaries ❌ Too broad

Recommendation: Use word-level lattices with proper Hindi tokenization (respects diacritics, number boundaries, compounds).



Model Agreement Strategy

When to Trust Models Over Reference:

Equivalence Validation:

Conclusion

This question evaluates your ability to:

Understand ASR evaluation limitations and their real-world impact

Design fair evaluation metrics that don't penalize valid alternatives

Handle linguistic complexity of Hindi (numbers, variants, synonyms)

Process multi-model outputs and extract consensus

Balance rigor and practicality in metric design

The lattice-based approach shifts evaluation from "punishing difference" to "rewarding correctness," which aligns with real speech perception where humans accept multiple valid transcriptions.